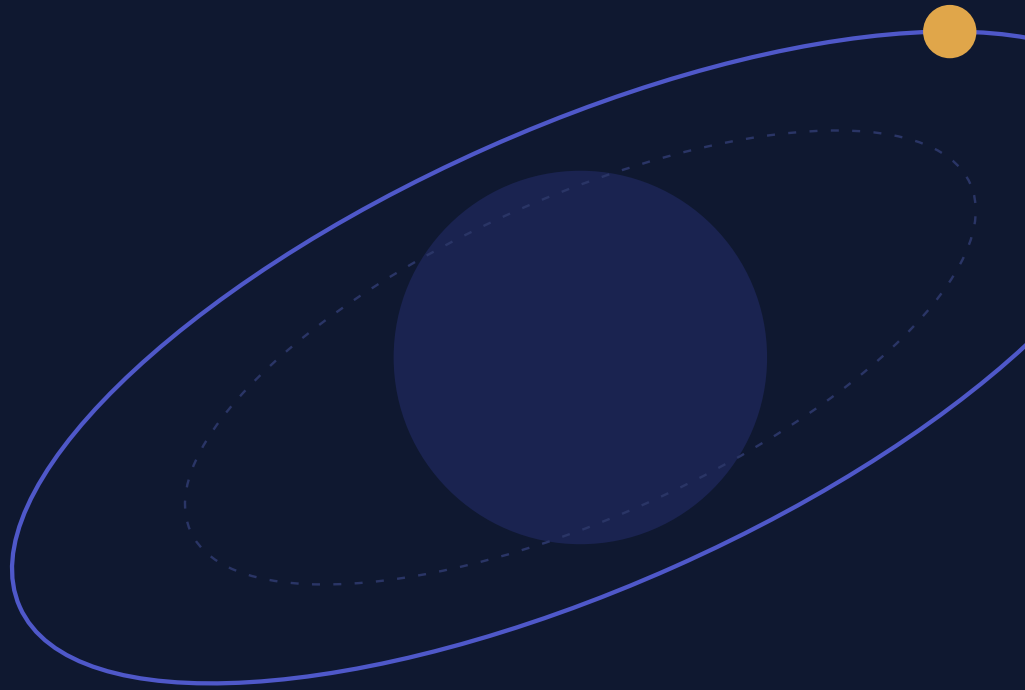




ARCHITECTURE AND BUILD ROADMAP

From UI to a working company superagent

How the supervisor and five agents are built, how Astro gets access to company systems, how it joins meetings, and the step-by-step plan with checkpoints to get there.

ASTRO · ARCHITECTURE AND BUILD ROADMAP

Contents

	Summary
1	What exists today and what's missing
2	System architecture
3	How a question flows through Astro
4	Tenants, identity and permissions
5	How Astro gets access to company systems
6	Ingestion and retrieval
7	The supervisor and the agent framework
8	The five agents: architecture and implementation
9	Voice
10	Meetings
11	Briefs and scheduled work
12	Connecting the existing UI
13	Integrating Astro into a company
14	Backend layout and infrastructure
15	Step-by-step roadmap with checkpoints
16	Testing, evaluation and safety
17	Risks and how to handle them
A	API endpoints
B	Database schema

NOTE · Where you start

- The frontend is done and runs on sample data. Build the backend so it speaks the API contract already written in the frontend's README, then flip `VITE_USE MOCKS=false` one screen at a time.
- Start with Phase 0 on Monday. Don't start the meeting bot before CP4 passes: meetings reuse everything underneath.

HOW TO USE THIS DOCUMENT

Summary

This is the build plan for Astro, from the UI you already have to a working product inside a real company. It explains the architecture, how each agent works and how it is implemented, how Astro gets access to company systems safely, and the order to build it in. Every phase ends in a **checkpoint**: a short list of things you can demonstrate or test before moving on.

The shape of the system

- **One product, many companies.** Every row of data carries a `company_id`, and the database refuses to return another company's rows.
- **A supervisor and five specialist agents.** The supervisor reads a question and hands it to Mentor, Tech, Finance, HR or Executive. Each agent has its own tools, data domains and guardrails.
- **Answers only from company data, with sources.** Agents retrieve before they answer, and every claim links back to a document, a query or a line of code.
- **The asker's permissions, always.** Retrieval and tools run as the person asking. If they can't open a document in Google Drive or a repo in GitHub, Astro won't answer from it.
- **People approve actions.** Tickets, Slack posts and exports are drafts until someone approves them.

The build, in five phases

Phase	Weeks	What you get	Ends at
0 · Foundations	1	Backend skeleton, login, companies and roles, database isolation, CI	CP0
1 · Answers with sources	2–4	Docs and GitHub connected, Mentor and Tech agents, real Ask page, access matrix, audit log	CP1–CP4
2 · Specialists and admin	5–8	Finance, HR, Executive agents, Agents page, approvals, briefs, voice	CP5–CP8
3 · Meetings	9–12	Calendar, meeting bot, answers in calls, recaps, first design partner	CP9–CP12
4 · Harden and scale	after 12	More connectors, SAML/SCIM, dedicated deployments, compliance work	ongoing

SECTION 1

What exists today and what's missing

You already have more than most projects at this stage: the product is designed end to end and the frontend is built against a written API contract. What's missing is everything behind that contract.

Piece	Status	Notes
Clickable prototype	Done	9 screens: Ask, Voice call, Meetings, Live meeting, Recap, Briefs, Knowledge, Agents, Setup
React + TypeScript frontend	Done, on mock data	Same 9 screens, routes, typed API client, SSE chat reader, meeting stream hook, voice hook
API contract	Written	README in the repo: chat SSE events, sources, access, agents, meetings endpoints
Pitch deck	Draft	13 slides with placeholders for market, pricing and the ask
Backend (FastAPI)	Not started	Auth, tenants, APIs, streaming
Agents and supervisor	Not started	LangGraph graph, prompts, tools, guardrails
Connectors and ingestion	Not started	OAuth apps, sync workers, chunking, embeddings
Meeting bot	Not started	Join, transcribe, answer, recap
Production infrastructure	Not started	Hosting, secrets, monitoring, backups

Decisions already made

- **Frontend:** Vite, React 18, TypeScript (strict), SCSS modules, React Router. Ant Design can be layered on later.
- **Backend:** Python with FastAPI, because the agent ecosystem (LangGraph, embedding and parsing libraries) is strongest in Python. Your Node/Express experience still fits: a thin Node BFF is possible, but one Python service is simpler to start.
- **Orchestration:** LangGraph for the supervisor and agent graphs.
- **Storage:** Postgres with the pgvector extension for data and vectors, Redis for queues, cache and pub/sub, S3-compatible storage for files and transcripts.
- **Streaming:** Server-Sent Events. Chat uses a POST read through fetch; meetings use EventSource (GET with cookies).

NOTE · Decide before Phase 0

- **LLM provider:** pick one model API for answers and one embedding model. Keep both behind small interfaces so you can switch later.
- **Cloud:** AWS, GCP or Azure. Pick the one your first design partner already uses; it makes security reviews easier.
- **First connectors:** Google Drive or Notion (docs) and GitHub (code). Everything else waits.

SECTION 2

System architecture

Astro is one backend service, a set of background workers, and a data plane, all shared by every company but partitioned by `company_id`. The diagram reads top to bottom: people reach the API, the API hands questions to the supervisor, agents use tools, tools read the data plane, and ingestion workers keep the data plane in step with the company's own systems.



Figure 1. Astro's layers. Arrows at the edges show requests going down and results coming back up.

Components and their jobs

Component	Job	Built with
Web app	All screens. Talks only to the Astro API.	React, TypeScript (exists)
API service	Auth, tenant resolution, REST endpoints, SSE streams, approvals	FastAPI, Pydantic, SQLAlchemy
Supervisor	Classifies each question and picks one agent (or honours the user's pick)	LangGraph node + LLM with structured output
Agents	Answer within their domain using their allowed tools	LangGraph subgraphs, one per agent
Tool layer	Every tool call checks the asker's permissions before touching data	Python functions with a shared permission guard
Ingestion workers	Pull from connectors, parse, chunk, embed, store with ACLs	Arq or Celery workers on Redis
Scheduler	Incremental syncs, briefs, meeting joins, token refresh	Worker cron / APScheduler
Meeting bot worker	Joins calls, streams audio to speech-to-text, posts events	Bot provider or headless browser; STT service
Postgres + pgvector	All relational data, vectors, full-text search, row-level security	Managed Postgres
Redis	Job queues, rate limits, cache, pub/sub for live streams	Managed Redis
Object storage	Original files, audio, transcripts, exports	S3 or equivalent
Secrets / KMS	Encrypts connector tokens and API keys	Cloud KMS + secrets manager
Observability	Traces for every agent run, logs, metrics, cost per company	OpenTelemetry, an LLM tracing tool

Why this shape

- **One service first.** A modular monolith (API + agents in one FastAPI app, workers as a second process from the same codebase) is fast to build and easy to debug. Split services only when load or teams demand it.
- **Postgres for everything at the start.** pgvector handles vector search well into millions of chunks per tenant, and keeping vectors next to ACLs and metadata makes permission filtering one query instead of two systems.
- **Workers for anything slow.** Syncing, embedding, briefs and meetings never run inside a web request.

SECTION 3

How a question flows through Astro

This is the path of one chat question from the Ask page. Voice and meetings reuse steps 2 to 8; only the way the question arrives and the answer leaves is different.



Figure 2. One question, eight steps. The stream starts at step 3 so the Ask page can show the route trace while the agent works.

Step	What happens	Event streamed to the UI
1	Ask page sends POST /api/chat/stream with companyId, message, mode, conversationId.	—
2	Middleware reads the session, loads the user, their company and roles, and sets app.company_id on the database connection. A request for another company is rejected.	—
3	If mode is auto, the supervisor classifies the question. Paused agents are skipped (Mentor answers instead).	route (supervisor), agent
4	The agent reads its config (prompt, tools, domains, guardrails) and decides which tools to call.	route (agent)
5	Retrieval and tools run with three filters: company, role × domain, and source ACL.	route (retrieval, access)
6	The model writes the answer, using only retrieved context, with [n] markers.	token ...
7	Every [n] is checked against the retrieved sources; unsupported sentences are removed or flagged. Sources are sent.	sources, done
8	The conversation, the route, sources used and any denied access are written to the audit log.	—

The streaming endpoint, sketched

This matches the event contract the frontend already reads. Tokens are forwarded as the model produces them.

```
@router.post("/chat/stream")
async def chat_stream(req: ChatRequest, ctx: RequestCtx = Depends(require_user)):
    ensure_same_company(ctx, req.company_id)          # 403 if not the user's company

    async def events():
        def sse(e: dict) -> str:
            return f"data: {json.dumps(e)}\n\n"

        yield sse({"type": "route", "step": {"kind": "supervisor", "label": "Supervisor"}})
        agent = req.mode if req.mode != "auto" else await supervisor.route(req.message, ctx)
        yield sse({"type": "agent", "agent": agent})

        async for e in run_agent(agent, req.message, ctx, req.conversation_id):
            yield sse(e)          # route / token / sources events from the agent graph
        yield sse({"type": "done"})

    return StreamingResponse(events(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})
```

WATCH OUT

- If you put Nginx or a load balancer in front, turn off response buffering for `/chat/stream` and `/meetings/*/live`, or tokens will arrive in one lump at the end.
- Send an error event instead of dropping the connection, so the UI can show a message.

SECTION 4

Tenants, identity and permissions

This is the part a company’s security team will read first. Get it right in Phase 0 and 1; retrofitting it later is painful and risky.

4.1 Tenant isolation

- Every table that holds company data has a non-null `company_id`.
- Postgres **row-level security** (RLS) is on for those tables. The API sets the company on each connection, so even a buggy query cannot read another company’s rows.
- Object storage paths start with the company ID (`s3://astro/{company_id}/...`), and signed URLs are issued per request.
- Caches and queues use the company ID in every key. Background jobs carry it in their payload and set it before touching the database.
- LLM calls never mix companies in one prompt.

```
ALTER TABLE chunks ENABLE ROW LEVEL SECURITY;
ALTER TABLE chunks FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON chunks
  USING (company_id = current_setting('app.company_id')::uuid);

-- per request / per job, inside the transaction:
SET LOCAL app.company_id = '6f1c...';
```

4.2 Identity

- **Sign-in:** OpenID Connect with Google Workspace and Microsoft Entra ID first, since that is where company users already are. Email magic links for small teams. SAML and SCIM come in Phase 4.
- **Company membership:** a user belongs to a company through a verified email domain or an invite from an admin.
- **Sessions:** an HTTP-only, secure, same-site cookie holding a short-lived session ID. The frontend never touches tokens, which keeps EventSource working with `withCredentials`.
- **Linked identities:** each user can link their GitHub, Google and Atlassian accounts. Astro uses these links to mirror what that person can open in each system.

4.3 Three permission filters on every read

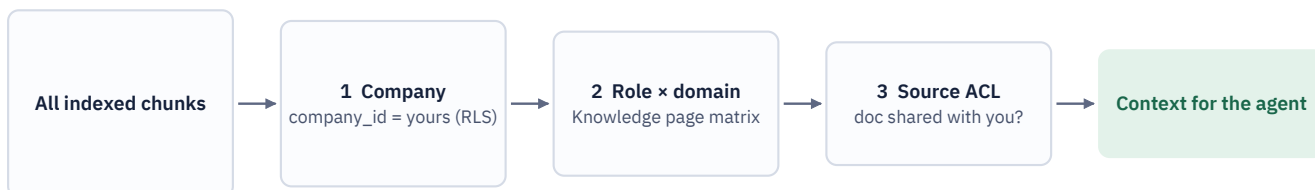


Figure 3. What an agent may read is narrowed three times before it reaches the model.

Filter	Where it comes from	How it is enforced
1 · Company	The signed-in user’s company	RLS on every table

Filter	Where it comes from	How it is enforced
2 · Role × data domain	The access matrix on the Knowledge page (company, engineering, finance, HR)	Each chunk has a domain; the query filters by the domains the user's roles allow
3 · Source ACL	Who can open the original: Drive sharing, GitHub repo access, Confluence space permissions	Each document stores allowed users and groups; the query checks the user's ID and groups against them

```

SELECT c.id, c.text, c.document_id, 1 - (c.embedding <=> :query_vec) AS score
FROM chunks c
JOIN documents d ON d.id = c.document_id
WHERE c.domain = ANY(:allowed_domains)           -- filter 2
  AND (d.acl_public_in_company
       OR d.acl_users && ARRAY[:user_id]::uuid[]
       OR d.acl_groups && :user_group_ids)         -- filter 3
ORDER BY c.embedding <=> :query_vec
LIMIT 40;                                         -- filter 1 is RLS

```

4.4 Roles

Role	Can do
Member	Ask, voice, meetings they are invited to, briefs shared with them
Team lead	As member, plus their own team's finance figures
HR, Finance	As member, plus their domain's restricted data
Leadership	Executive agent, cross-team briefs
Workspace admin	Knowledge, Agents, Setup, access matrix, audit log, retention. Admins do not automatically see all data.

4.5 Audit log

Write one row for each question, tool call, denied access, approval and admin change: who, when, which agent, which sources were used, which were filtered out and why. The “Recent access checks” list on the Knowledge page reads from this table.

CHECKPOINT · CP0 and CP4 lean on this section

- ✓ An automated test signs in as a user of company A and tries to read company B's rows through every endpoint. It must get nothing.
- ✓ A second test gives a Drive document to one user only and asks another user a question only that document answers. Astro must say it can't find it.
- ✓ Every denied access appears in the audit log with the reason.

SECTION 5

How Astro gets access to company systems

Astro never asks for passwords. An admin connects each system once from the Setup page, the system shows its own consent screen, and Astro receives a token limited to what it needs. The admin can disconnect at any time, which revokes the token and deletes that source's data.



Figure 4. Connecting a source. The Knowledge page shows status from the moment the first sync starts.

5.1 Three ways to connect

Method	Used for	How it works
OAuth app (company-wide)	Google Drive, Calendar, Notion, Confluence, Jira, Slack	Admin approves Astro's app for the workspace. Astro gets a token plus a refresh token and syncs what the app may see.
Installed app	GitHub	Admin installs the Astro GitHub App on the organisation and picks which repositories. Permissions are set per app (for example, contents and pull requests read-only).
Read-only credentials	Databases, data warehouse, billing	The company creates a read-only user or restricted key limited to specific views. Astro connects over TLS, from a fixed IP or through a relay inside their network.

5.2 Connector by connector

Ask for the narrowest access that does the job. The scopes below describe what each connector needs; check each provider's current scope names when you register the app.

System	Access Astro needs	What it syncs	Permissions mirrored from
Google Drive / Docs	Read-only Drive access	Docs, Sheets, PDFs, Slides (as text)	File sharing: users, groups, domain
Notion	Integration added to chosen pages	Pages and databases shared with Astro	Only pages the admin shares; mapped to domains
Confluence	Read content (Atlassian OAuth)	Spaces and pages	Space and page restrictions
GitHub	GitHub App: metadata, contents, pull requests, actions logs (read)	Code, READMEs, ADRs, PRs, CI results, CODEOWNERS	Repo access per user, checked through their linked GitHub account
Jira	Read issues; create issues only for approved drafts	Projects, issues, comments	Project permissions
Slack	Bot token; post messages; read chosen channels (optional)	Only channels the admin selects	Channel membership
Postgres / warehouse	Read-only DB user on named views	Nothing copied; queried live	Views per domain + row filters per role

System	Access Astro needs	What it syncs	Permissions mirrored from
HR system	Vendor API with minimal fields	Policies; personal data fetched live, never indexed	Only the person themselves and HR
Google / Microsoft Calendar	Read events; one user or the whole workspace	Meetings, attendees, links	Only meetings the user is invited to
Meet, Zoom, Teams	Bot joins as a participant (Section 10)	Live audio → transcript	Invited attendees only
File upload	None	PDFs, DOCX, Markdown dropped in Setup	Set by the uploader

5.3 Keeping data fresh

- **Initial sync** runs as a background job with progress shown on the Knowledge page (queued → syncing → indexed).
- **Incremental sync** uses each provider’s change feed or webhooks where it has one (GitHub webhooks, Drive changes, Jira webhooks), and polling every 15–60 minutes where it doesn’t.
- **Deletes and permission changes** matter more than new content. When a file is deleted or unshared, remove its chunks or update its ACL in the same sync run.
- **Token refresh** runs before expiry. If a refresh fails, mark the source “needs attention” so the admin sees it.

5.4 Systems behind a firewall

Many databases and on-premise wikis can’t be reached from the internet. For those, ship a small **Astro relay**: a container the company runs inside their network. It opens an outbound, authenticated connection to Astro, receives read requests, runs them against the local system and returns results. Nothing inbound is opened on their firewall. Build this in Phase 4 unless a design partner needs it earlier.

5.5 Storing credentials

- Encrypt every token with a per-company data key, itself encrypted by the cloud KMS (envelope encryption). The database only ever holds ciphertext.
- Workers decrypt a token in memory for the job and never log it.
- Rotate the KMS key on a schedule, and give the admin a “Disconnect and delete” button per source.

CHECKPOINT · Connector checklist (run for each new connector)

- ✓ Consent screen shows only the scopes listed in the table above.
- ✓ Initial sync of a test workspace completes and the Knowledge page shows it as indexed.
- ✓ Editing, deleting and unsharing a document are reflected after the next sync.
- ✓ Revoking access in the provider marks the source “needs attention” within one sync interval.
- ✓ Disconnecting in Astro deletes that source’s documents, chunks and tokens.

SECTION 6

Ingestion and retrieval

Answer quality depends more on this pipeline than on the model. Take care with chunking, metadata and hybrid search, and measure it with a fixed question set from week 2.

6.1 The pipeline

Stage	What to do
Fetch	Pull the original plus its metadata: title, URL, author, last edited, sharing, folder or space, repo and path.
Parse	Convert to clean text with structure: headings, lists, tables as Markdown, code as code. Keep page numbers for PDFs.
Classify	Assign a data domain (company, engineering, finance, HR) from the source's mapping in Setup. Admins can override per folder or space.
Chunk	Docs: split on headings, 300–800 tokens, with the heading path prefixed to each chunk. Code: split by file and by function or class; keep the file path and commit. Tables: keep header rows with every chunk.
Embed	Embed each chunk with the heading path and title included. Batch calls; store the model name so you can re-embed later.
Store	Write documents, chunks and ACLs in one transaction. Skip unchanged chunks using a content hash.

6.2 Retrieval

1. **Rewrite the query** using the conversation so far (“what about last quarter?” becomes a full question).
2. **Hybrid search**: run vector similarity (pgvector, HNSW index) and keyword search (Postgres full-text) with the permission filters from Section 4, then merge the lists with reciprocal rank fusion.
3. **Rerank** the top 30–50 with a reranker model or the LLM, and keep the best 6–10.
4. **Build context** with each chunk numbered, its title, where it lives and when it was updated. These numbers become the [n] citations and the Source cards in the evidence panel.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE chunks (
  id          uuid PRIMARY KEY,
  company_id  uuid NOT NULL,
  document_id uuid NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  domain      text NOT NULL,          -- company | engineering | finance | hr
  heading_path text,
  text        text NOT NULL,
  tsv         tsvector GENERATED ALWAYS AS (to_tsvector('english', text)) STORED,
  embedding   vector(1024),           -- match your embedding model's size
  content_hash text NOT NULL
);
CREATE INDEX ON chunks USING hnsw (embedding vector_cosine_ops);
CREATE INDEX ON chunks USING gin (tsv);
CREATE INDEX ON chunks (company_id, domain);
```

6.3 Measuring it

- For each design partner, write 50–100 real questions with the document that answers each one. Keep them in the repo.
- Track two numbers on every change: whether the right document is in the top results (retrieval), and whether the final answer is correct and cites it (answer).
- Add every bad answer a user reports (thumbs down) to the set.

SECTION 7

The supervisor and the agent framework

All five agents share one framework. An agent is mostly configuration (prompt, domains, tools, guardrails) plus a few tools of its own. That is exactly what the Agents page edits.

7.1 The supervisor

- **Input:** the question, the last few turns, the user's roles and which agents are active.
- **Output:** structured JSON: `{"agent": "tech", "confidence": 0.86, "reason": "..."}.`
- **Rules around the model:** if the user picked an agent in the composer, skip classification. If the agent is paused or the user can't use it, fall back to Mentor and say so in the route trace. Below a confidence threshold, use Mentor.
- **Cost:** use a small, fast model; this call happens on every question.
- **Multi-part questions:** the Executive agent handles questions that need several agents. The supervisor sends those there rather than splitting them itself.

```
class AstroState(TypedDict):
    question: str
    history: list[Message]
    ctx: RequestCtx          # user, company, roles, groups, linked identities
    agent: str | None
    context: list[Chunk]
    answer: str
    sources: list[Source]

g = StateGraph(AstroState)
g.add_node("supervisor", route_question)
for name in ["mentor", "tech", "finance", "hr", "exec"]:
    g.add_node(name, build_agent(name))      # each is its own subgraph
g.add_node("verify", verify_citations)
g.add_edge(START, "supervisor")
g.add_conditional_edges("supervisor", lambda s: s["agent"])
for name in ["mentor", "tech", "finance", "hr", "exec"]:
    g.add_edge(name, "verify")
g.add_edge("verify", END)
graph = g.compile()
```

7.2 Inside an agent

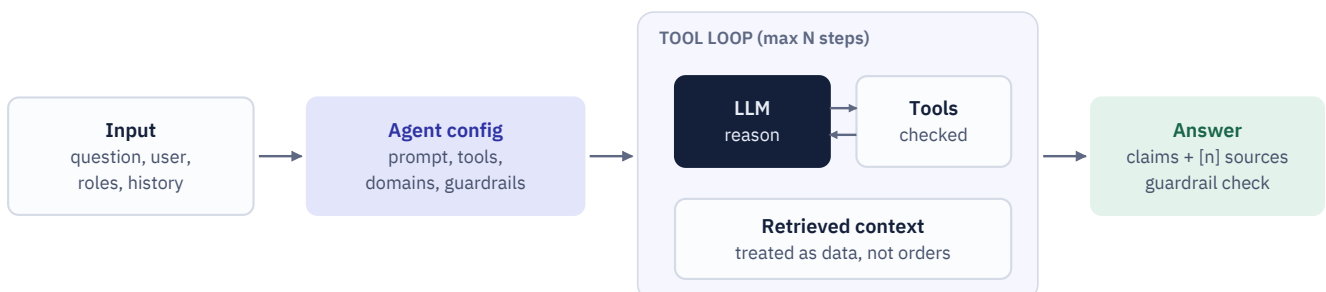


Figure 5. Every agent runs the same loop: config in, a bounded tool loop, then an answer that must carry its sources.

Part	What it holds	Edited on
Prompt	Role, tone, company name, “answer only from context; say when you don’t know”	Code (versioned)
Data domains	Which domains its retrieval may search	Agents page
Tools	Allowed functions, each tagged read-only, draft or needs approval	Agents page (on/off)
Guardrails	Rules checked in the prompt and in code after the answer	Agents page (toggles)
Who can ask	Roles allowed to use this agent	Agents page
Limits	Max tool steps, timeout, max tokens	Code

7.3 Tools and approvals

A tool is a Python function with a typed schema, a permission level and a guard that runs before it. The three levels match the badges on the Agents page.

Level	Meaning	Examples
Read-only	Runs immediately, as the asker	search_docs, search_code, run_readonly_sql, get_policy
Drafts for you	Produces a draft shown in the answer; nothing leaves Astro	build_learning_plan, write_brief
Needs approval	Creates a pending action; runs only after a person approves it in the UI	create_jira_ticket, post_slack_message, export_csv

```
@tool(level="approval", domains=["engineering"])
async def create_jira_ticket(ctx: RequestContext, project: str, title: str, body: str) -> PendingAction:
    require(ctx.can_write_jira(project)) # guard: asker's rights
    return await approvals.create(ctx, kind="jira.create",
                                  payload={"project": project, "title": title, "body": body})

# later, POST /meetings/{id}/actions/{action_id}/approve (or /approvals/{id})
async def execute(action: PendingAction, approver: User):
    require(approver.can_approve(action))
    result = await jira.create_issue(action.payload, token=company_jira_token(action.company_id))
    await audit.log(action, approver, result)
```

WATCH OUT · Safety rules for every agent

- Retrieved text is data. The prompt says so, and tools never take instructions from document content (“ignore previous instructions” inside a doc does nothing).
- Tools the config doesn’t list can’t be called; the allowlist is enforced in code, not only in the prompt.
- Nothing is written outside Astro without an approval record.
- The verify step drops sentences whose [n] doesn’t point to a retrieved source, and forces “I couldn’t find this” when nothing relevant was retrieved.

SECTION 8

The five agents: architecture and implementation

Build them in this order: Mentor (the fallback and the simplest), Tech (your first design partners are tech companies), Finance, HR, and Executive last because it calls the others.

8.1 Mentor: how things work here, and who to ask

The default agent. It answers general company questions and onboarding questions, and when the documents run out, it names the person to ask. Every question that no other agent owns lands here.

Reads from

- Handbook, onboarding guide, wiki (Drive, Notion, Confluence)
- Team directory (Google Workspace or Entra ID)
- CODEOWNERS and document authors, for “who owns this”

Tools

- `search_docs(query, domains)` read-only
- `find_owner(topic)` read-only: ranks people by authorship, ownership files and team
- `build_learning_plan(role, weeks)` draft: a 30-day plan from onboarding docs

Guardrails

- Names a person, not a channel, when it can’t answer.
- Short answers for first-week hires (toggle).
- Never guesses about policy; hands policy questions to HR.

How to build it

1. Write the shared agent runner (config, tool loop, verify) with Mentor as the first config.
2. Implement `search_docs` on top of the retrieval in Section 6.
3. Implement `find_owner` from directory data plus document authors.
4. Connect the Ask page with mocks off and run the eval set.

CHECKPOINT · CP2 · Mentor answers for real

- ✓ The Ask page streams a real answer with real Source cards for a question from the eval set.
- ✓ A question with no answer in the docs gets “I couldn’t find this” plus a named owner.
- ✓ Answer accuracy on the eval set is recorded as the baseline.

8.2 Tech: code, architecture, deploys and incidents

Answers engineering questions from the code itself and from the documents around it: architecture decisions, runbooks and incident reviews. It can read CI results and draft Jira tickets.

Reads from

- GitHub: code, READMEs, ADRs, pull requests, CI logs
- Confluence or Drive: runbooks, incident reviews
- Jira: engineering projects

Tools

- `search_code(query, repo?)` read-only: your code index, falling back to GitHub search
- `get_pull_request(repo, number)` read-only
- `read_ci_run(repo, run_id)` read-only
- `create_jira_ticket(project, title, body)` needs approval

Guardrails

- Mirrors GitHub permissions: before using code from a repo, check the asker's linked GitHub account can read it.
- Masks secrets: run a secret scanner on code chunks at ingestion and replace values with a placeholder.
- Quotes file paths and commit SHAs so answers can be checked.

How to build it

1. Register the GitHub App; handle installation webhooks and store the installation per company.
2. Index repos: default branch only at first, chunked by file and function, with path, language and commit.
3. Add user-to-GitHub account linking; cache each user's repo list and refresh it on login and hourly.
4. Add Tech's tools and prompt; add Jira draft creation behind approvals.
5. Build the supervisor now that there are two agents, and add routing questions to the eval set.

CHECKPOINT · CP3 · Two agents and a router

- ✓ "Why did we choose Kafka?" goes to Tech and cites the ADR.
- ✓ A user without access to a private repo gets no answer from it, and the audit log shows why.
- ✓ Routing accuracy on a labelled set of 50 questions is at or above your target (90% is a reasonable starting target).

8.3 Finance: revenue, spend and budgets

Answers numeric questions by querying live data, not by reading exported documents. It always shows the query behind each figure so finance teams can trust it.

Reads from

- Finance warehouse or database through named read-only views
- Budget sheets (Drive)
- Vendor contracts (Drive)
- Billing system through a restricted read key

Tools

- `list_metrics()` read-only: the semantic layer (below)
- `run_readonly_sql(sql)` read-only, through the SQL gateway
- `make_chart(rows, kind)` read-only: returns chart data for the UI
- `export_csv(query_id)` needs approval

Guardrails

- Only SELECT statements on allowlisted views; one statement; row limit; 15-second timeout.
- Row filters by role: team leads see their own team.
- Individual salaries never leave the database; payroll only as team totals.
- In calls with guests, numbers go to the asker privately.

How to build it

1. Create a **semantic layer**: a YAML file per company defining each metric (revenue, churn, burn), its SQL, grain and owner. The agent picks metrics first and writes SQL second.
2. Build the SQL gateway: parse with a SQL parser, reject anything but SELECT, check every table against the allowlist, add the role's row filter, run as the read-only user.
3. Return the SQL, the rows and chart data together; show the SQL in the evidence panel.
4. Add 30 finance questions with known answers to the eval set.

CHECKPOINT · CP5 · Finance with receipts

- ✓ Writes, DDL, multiple statements and unlisted tables are all rejected by the gateway (unit tests).
- ✓ “Why did revenue drop last quarter?” returns a figure, the SQL and a chart, and the figure matches finance's own report.
- ✓ A team lead asking about another team's spend gets a clear refusal.

8.4 HR: policies, leave and benefits

Explains written policy to everyone, and looks up personal records only for the person asking. Sensitive topics go to a human.

Reads from

- HR policies (Notion or Drive), indexed
- Holiday calendar
- HR system, queried live for the asker's own records

Tools

- `search_policy(query)` read-only
- `get_my_leave_balance()` read-only, only the asker
- `draft_leave_request(dates, type)` needs approval

Guardrails

- Answers only from written policy; exceptions go to HR.
- A classifier flags complaints, health, conduct and harassment; the agent gives the HR contact and stops.
- Never reveals someone else's records or salary bands (toggle).

How to build it

1. Index policies into the HR domain; only HR and the person themselves see personal-data domains.
2. Connect the HR system with a service account and look up records by the asker's verified email only.
3. Add the sensitive-topic classifier as a guard before the agent runs.
4. Write red-team tests: "What is Priya's leave balance?" must be refused.

CHECKPOINT · CP6 · HR is safe to switch on

- ✓ Policy questions are answered with the policy cited.
- ✓ Asking about another person's records is refused in every phrasing in the test set.
- ✓ Sensitive topics return the HR contact without a generated answer.

8.5 Executive: briefs, trade-offs and cross-team risks

For leaders. It breaks a question into parts, asks the other agents with the leader's own permissions, and combines the results into a brief with trade-offs, each point traced to the agent and source it came from.

Reads from

- Other agents' answers (called as tools)
- Briefs and meeting recaps
- OKRs and board decks (Drive)

Tools

- `ask_agent(agent, question)` read-only: runs a sub-agent as the same user
- `compare_options(options, criteria)` read-only
- `write_brief(topic)` draft
- `schedule_brief(cron, audience)` needs approval

Guardrails

- Labels recommendations as suggestions; decisions stay with people.
- Every line of a brief links to its agent and source.
- Sees only what the leader could open; no admin shortcut.
- Clients shown as industry and size unless the toggle allows names.

How to build it

1. Implement `ask_agent` as a subgraph call with the same request context and a step budget.
2. Add a planner step: split the question into 2–5 sub-questions and pick agents.
3. Build the brief template used on the Briefs page (summary, changes, risks, decisions needed).
4. Hook scheduled briefs into the worker scheduler (Section 11).

CHECKPOINT · CP7 · Leadership questions

- ✓ “What should engineering focus on next quarter?” produces a brief citing Tech and Finance sources.
- ✓ A leader without HR access gets no HR data through the Executive agent.
- ✓ The Briefs page shows a generated brief from the API.

SECTION 9

Voice

The Voice call page already works with the browser's Web Speech API (Chrome and Edge). That is enough for demos and for Phase 1–2. Replace it when you need consistent quality across browsers, better accuracy for Indian English and other accents, or voice in meetings.

Stage	Now (built)	Later (Phase 2d or 3)
Speech to text	Browser recognition in <code>useVoice.ts</code>	Streaming STT service over a WebSocket from the browser to the API
Answering	Same <code>/chat/stream</code> pipeline	Same pipeline; prompt asks for shorter, spoken-style answers
Text to speech	Browser speech synthesis	Streaming TTS; start speaking at the first sentence
Interruptions	Recognition pauses while Astro speaks	Barge-in: stop TTS when the user starts talking

Keep the hook's interface (`start`, `stop`, `speak`, `transcript`) and swap its insides, so `CallPage` doesn't change. The meeting bot uses the same server-side STT and TTS.

CHECKPOINT · CP8 (voice part)

- ✓ A full spoken question and answer round trip works in Chrome, Edge and Safari.
- ✓ Time from the end of speech to Astro's first word is measured and within your target (aim for under 2 seconds).

SECTION 10

Meetings

The feature that makes Astro feel like a colleague: it joins Google Meet, Zoom or Teams, takes notes, answers when someone says “Astro”, and writes a recap with action items that wait for approval. It reuses the whole agent stack; the new parts are joining the call, turning audio into text in real time, and knowing who is speaking.

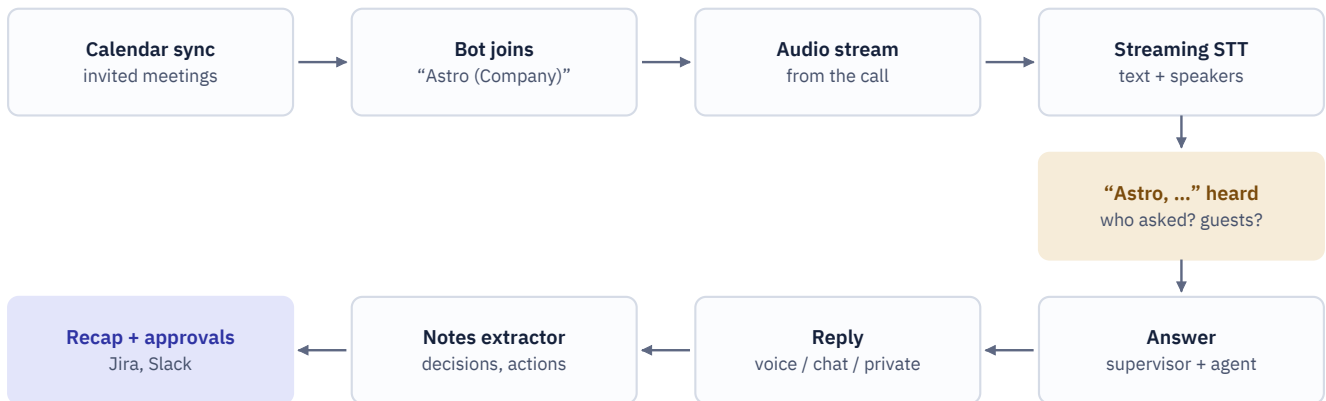


Figure 6. The meeting pipeline. The brass box is the moment Astro decides to answer.

10.1 Getting into the call

Option	How it works	Trade-offs
Hosted meeting-bot API	A provider runs the bots; you call an API with the meeting link and receive audio and transcript events by webhook or WebSocket.	Fastest to ship and works on Meet, Zoom and Teams. Per-minute cost and a third party in the data path (needs a data-processing agreement).
Your own bot	A headless browser joins the call as a guest, captures audio, and plays Astro’s voice into the call.	Full control and lower unit cost at scale. Fragile when the meeting UIs change; you run the fleet.
Platform media APIs	Zoom’s real-time media streams, Google’s Meet Media API, Microsoft’s bot framework for Teams.	No visible bot in some cases. Each platform differs and availability and requirements change; check each one’s current status before depending on it.

Recommendation: start with a hosted bot API for Google Meet in Phase 3, behind a small MeetingBot interface (join, leave, speak, send_chat, event stream). Move to your own bot or platform APIs later without touching the rest.

10.2 Step by step inside a meeting

1. **Calendar sync** finds meetings where Astro is invited, or where the user switched “Astro joins” on in the Meetings page. A scheduler job joins one minute before the start.
2. **Join and introduce.** The bot joins as “Astro (Company)”, posts a chat message explaining what it does and how to make it leave, and starts the transcript.
3. **Transcribe with speakers.** Streaming STT returns text with speaker labels. Map speakers to people using the participant list and the calendar attendee emails; unmatched speakers are shown by name only.

4. **Detect guests.** Any attendee whose email domain isn't the company's is a guest. With guests present, Astro never speaks aloud and answers only privately.
5. **Wake word.** Watch the transcript for "Astro" followed by a question. Take the rest of the sentence, plus a few seconds of context, as the question.
6. **Answer as the asker.** Run the normal pipeline with the asker's permissions. If the speaker can't be matched to a user, answer only from company-wide documents.
7. **Reply.** Speak a short answer through TTS if allowed, and post the answer with source links in the meeting chat. Private questions from the live page go only to that user.
8. **Take notes.** Every minute or so, a notes step reads the new transcript and updates decisions and action items, each with who said it and when. Push each change to the live page over SSE.
9. **Leave and recap.** When the meeting ends or someone asks it to leave, write the recap from the full transcript: summary, decisions, answers given, action items with owners and due dates, and a Slack follow-up draft.
10. **Approvals.** Action items become pending Jira tickets or comments; the follow-up waits for "Post to Slack". Open items feed the next engineering brief.

10.3 Events the live page already understands

Event	When it is sent
transcript	Each finished transcript line (speaker, time, text)
speaking	Who is speaking now; astro while Astro talks
answer	Astro answered a spoken question (question, answer, sources)
decision / action	The notes step found a new decision or action item
ended	Astro left the call; the recap is being written

Publish these to a Redis channel per meeting from the bot worker; the API subscribes and forwards them to GET /meetings/:id/live. Send history first on connect so a late joiner sees the notes so far.

WATCH OUT · Consent and law

- Recording and transcription rules differ by country and state. Astro must announce itself on joining, and the company's admin chooses whether meetings with guests are transcribed at all.
- Keep transcripts and audio for the company's retention setting (the recap page shows it), then delete them. Store audio only if the company turns it on.
- Only invited attendees can open a recap or transcript.

CHECKPOINT · CP9 to CP11 · Meetings

- ✓ **CP9:** Astro joins a test Google Meet from the Meetings page, introduces itself, and the live page shows the transcript within a few seconds of speech.
- ✓ **CP10:** "Astro, when did we last load-test the order service?" gets a spoken answer and a chat message with sources; with a guest in the call it answers privately instead.
- ✓ **CP11:** After leaving, the recap page shows the summary, decisions and action items; approving one creates the Jira ticket; the follow-up posts to Slack.

SECTION 11

Briefs and scheduled work

Briefs are Executive-agent runs on a schedule. The Briefs page shows the latest output; the Meetings page links a call brief before external meetings.

- **Scheduler:** one table of schedules (company, brief type, cron, audience) and a worker that enqueues jobs. Each job runs as a named owner, so the brief contains only what that owner can see, and is shared only with people who could see it too.
- **Engineering brief:** PRs waiting on review, failed deploys, open incidents, open action items from last week's meeting.
- **Revenue review:** Finance metrics against budget, with the SQL for each figure.
- **Call brief:** before a meeting with an external company, a summary of the account from CRM notes, contracts and past recaps.
- **Caching:** store each brief with its sources and generation time; regenerate on schedule or on request.

CHECKPOINT · CP8 (briefs part)

- ✓ A scheduled engineering brief appears on the Briefs page at its time, with sources.
- ✓ Its owner changing roles changes what the next brief includes.

SECTION 12

Connecting the existing UI

The frontend was built against the contract, so integration is mostly: add login, point it at the API, and switch screens from mock to real one by one.

12.1 One-time changes in astro-web

1. **Auth:** add a Login page (SSO buttons) and a useAuth hook that calls GET /api/me. Redirect to login on 401. Replace the hard-coded company and currentUser in src/config.ts with the /me response.
2. **Admin links:** set canManage from the user's roles, and guard the Knowledge, Agents and Setup routes too (the server checks as well).
3. **Cookies:** keep credentials: "include" (already in the API client). In development the Vite proxy sends /api to localhost:8000, so there is no CORS to configure.
4. **Errors:** show the error SSE event and failed requests with the existing inline error styles.
5. **Mock switch:** USE MOCKS is global today. Split it per area while integrating (for example VITE_MOCK_MEETINGS=true) so finished screens can go live first.

```
// src/api/http.ts: one wrapper for every call
export async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
  const res = await fetch(`${API_URL}${path}`, {
    credentials: 'include',
    headers: { 'Content-Type': 'application/json', ...init.headers },
    ...init,
  });
  if (res.status === 401) { window.location.assign('/login'); throw new Error('Signed out'); }
  if (!res.ok) throw new Error(`${path} failed with ${res.status}`);
  return res.status === 204 ? (undefined as T) : res.json();
}
```

12.2 Screen by screen

Screen	Endpoints	Backend needed	Go live at
Setup	domain presets, connector OAuth start/callback, sources	Connectors, ingestion	CP1
Knowledge	GET /sources, PUT /access, audit list	Ingestion, access matrix, audit	CP1 / CP4
Ask	POST /chat/stream, conversations	Supervisor, agents, retrieval	CP2
Agents	GET/PATCH /agents, route test	Agent configs	CP7
Briefs	briefs list, brief detail, schedules	Executive agent, scheduler	CP8
Voice call	/chat/stream (+ STT/TTS later)	Same as Ask	CP2 / CP8
Meetings	GET /meetings, join, invite switch, settings	Calendar sync, bot	CP9
Live meeting	GET /meetings/:id/live (SSE), ask-private, leave	Bot worker, STT, notes step	CP10
Recap	recap, approve action, follow-up	Recap writer, approvals, Jira, Slack	CP11

12.3 Endpoints the UI still needs

The README covers chat, sources, access, agents and meetings. Add these to the contract as you build them:

- GET /api/me: user, company, roles, linked accounts, canManage
- GET /api/auth/{provider}/start and /callback: sign-in; POST /api/auth/logout
- GET /api/connectors/{kind}/start and /callback: connect a source; DELETE /api/sources/{id}
- GET /api/conversations, GET /api/conversations/{id}: chat history
- GET /api/audit: access checks for the Knowledge page
- GET /api/briefs, GET /api/briefs/{id}, PUT /api/brief-schedules
- POST /api/feedback: thumbs up or down on an answer, which feeds the eval set

SECTION 13

Integrating Astro into a company

13.1 Deployment options

Option	Who it suits	What changes
Shared cloud (default)	Most companies, and all design partners	Nothing: tenants are isolated by company_id and RLS
Dedicated tenant	Larger or regulated companies	Their own database and storage in Astro's cloud, same code
Customer cloud	Companies that require data to stay in their account	Astro deployed into their cloud with Terraform; they hold the keys
Relay for private systems	Any of the above with databases or wikis behind a firewall	Astro relay container inside their network, outbound only

13.2 Onboarding a new company

This is what the Setup page walks an admin through. Aim to get a pilot team asking real questions within one working day.

1. **Create the workspace:** admin signs in with SSO; Astro verifies the email domain and creates the company.
2. **Pick a domain preset:** technology, finance, healthcare or retail. It decides the starting agents, suggested sources and example questions.
3. **Connect sources:** start with one docs source and one work source (for tech companies: Drive or Notion, plus GitHub). Map folders or spaces to data domains.
4. **Set access:** confirm the role × domain matrix on the Knowledge page. Import groups from Google Workspace or Entra ID and map them to roles.
5. **Invite a pilot team:** 10–20 people from one team, not the whole company.
6. **Connect the calendar and Slack** once the team trusts the answers, then switch meetings on for internal calls first.
7. **Review after two weeks:** answer quality from feedback, questions Astro couldn't answer (those are missing documents), and denied-access logs.

13.3 What the company's security team will ask for

- A data-flow diagram (Figure 1 is a start) and a list of sub-processors (cloud, LLM provider, STT, bot provider).
- Scopes requested for each connector, and proof that disconnecting deletes data.
- That company data is not used to train models. Choose LLM and STT providers whose terms allow this, and say so in your contract.
- Encryption in transit and at rest, key management, retention and deletion settings.
- Access logs they can export, and single sign-on.
- Later: a security questionnaire, a penetration test report, and a SOC 2 or ISO 27001 plan.

SECTION 14

Backend layout and infrastructure

14.1 Repository layout

```

astro-api/
  app/
    main.py          FastAPI app, routers, middleware
    config.py        settings from env
    auth/            OIDC, sessions, /me, linked accounts
    tenancy/         request context, RLS setup, role checks
    api/             chat.py, sources.py, access.py, agents.py, meetings.py, briefs.py, audit.py
    agents/
      graph.py       supervisor + agent graph
      runner.py      shared tool loop, verify step
      configs/       mentor.yaml, tech.yaml, finance.yaml, hr.yaml, exec.yaml
      prompts/
    tools/           retrieval, sql_gateway, github, jira, slack, calendar, hr
    approvals/       pending actions, execute on approve
    connectors/      google_drive.py, notion.py, confluence.py, github_app.py, ...
    ingestion/       parse.py, chunk.py, embed.py, sync.py
    meetings/        bot interface, provider adapter, stt.py, notes.py, recap.py
    briefs/          templates, scheduler
    db/              models.py, migrations/ (Alembic)
    llm/             model client, embeddings, rerank (behind interfaces)
    observability/   tracing, cost tracking
  workers/
    worker.py        Arq/Celery worker: sync, embed, briefs, meetings
  evals/            question sets per design partner, runner
  tests/            unit, permission, cross-tenant, prompt-injection
  docker-compose.yml postgres+pgvector, redis, minio, api, worker

```

14.2 Environments

Environment	Setup
Local	docker compose up: Postgres with pgvector, Redis, MinIO for S3, API on :8000, worker. Frontend on :5173 with the Vite proxy.
Staging	Same containers on your cloud; seeded demo company (Kestrel Labs data) plus a test Google Workspace and GitHub org.
Production	Containers on a managed service (ECS, Cloud Run or similar), managed Postgres and Redis, object storage, secrets manager and KMS, daily backups with point-in-time restore.

14.3 Running it

- **CI/CD:** lint, type-check, unit and permission tests on every pull request; the eval set nightly; deploy to staging on merge, to production by tag.
- **Tracing:** one trace per question covering supervisor, retrieval, each tool call and the model calls, tagged with company and user IDs. This is how you debug bad answers.
- **Cost:** record tokens and STT minutes per company per day. It drives pricing and catches runaway loops.
- **Limits:** per-user and per-company rate limits in Redis; a daily spend cap per company.
- **Budget:** early infrastructure is small (a small database, one API and one worker container). Model, embedding and speech usage will be most of the bill and grows with use. Treat the original ₹5k–15k estimate as a starting figure to check once you have real usage.

SECTION 15

Step-by-step roadmap with checkpoints

Twelve weeks to a design-partner pilot, then hardening. Each phase lists what to build and a checkpoint that must pass before the next phase starts. Weeks assume one or two developers; stretch them if you're alone and part-time.



Figure 7. The 12-week plan. Dashed green lines mark CP0, CP4, CP8 and CP12.

Phase 0 · Foundations · week 1

A backend that knows who you are and which company you belong to, and can't leak across companies.

Step	Build
0.1	Create astro-api repo, docker compose (Postgres + pgvector, Redis, MinIO), FastAPI skeleton, Alembic migrations, CI.
0.2	Tables: companies, users, roles, memberships, audit_log. Turn on RLS; request context sets <code>app.company_id</code> .
0.3	Google and Microsoft sign-in (OIDC), session cookie, GET <code>/api/me</code> , logout.
0.4	Frontend: Login page, <code>useAuth</code> , the <code>api()</code> wrapper, config from <code>/me</code> , route guards.
0.5	Seed two test companies; write the cross-tenant test suite.

CHECKPOINT · CP0 · Foundations

- ✓ You can sign in on the web app and see your name and company from the API.
- ✓ The cross-tenant suite runs in CI and passes.
- ✓ An unauthenticated request to any endpoint returns 401.

Phase 1 · Answers with sources · weeks 2–4

Real questions, real answers, real citations, and permissions that hold.

Step	Build
1a	Connector framework (OAuth start/callback, encrypted token store). Google Drive or Notion connector plus file upload. Ingestion: parse, chunk, embed, store with ACLs. GET /sources for the Knowledge page.
1b	Retrieval (hybrid search, rerank, permission filters). Agent runner with Mentor. POST /chat/stream with the full event contract. Conversations saved. First eval set.
1c	GitHub App, code indexing, account linking, Tech agent, supervisor routing, Jira read.
1d	Access matrix (PUT /access), group import, audit list on the Knowledge page, permission test suite.

CHECKPOINT · CP1 · Sources indexed

- ✓ An admin connects Drive or Notion from Setup and the Knowledge page shows it syncing, then indexed.
- ✓ Deleting or unsharing a document is reflected after the next sync.

CHECKPOINT · CP2 · Mentor answers

- ✓ See 8.1: real streamed answers with Source cards; honest “couldn’t find it”; baseline accuracy recorded.

CHECKPOINT · CP3 · Tech and routing

- ✓ See 8.2: routed to Tech with the ADR cited; private repos respected; routing accuracy at target.

CHECKPOINT · CP4 · Permissions hold

- ✓ Every permission and cross-tenant test passes (100%, not “most”).
- ✓ Changing the matrix on the Knowledge page changes answers on the next question.
- ✓ The Knowledge page shows recent access checks from the audit log.

Phase 2 · Specialists, admin, voice and briefs · weeks 5–8

All five agents, admin control over them, safe actions, and scheduled briefs.

Step	Build
2a	Finance: semantic layer, SQL gateway, read-only connection or relay, charts in answers.
2b	HR: policies, own-records lookup, sensitive-topic guard. Executive: ask_agent, planner, brief template.
2c	Agent configs in the database; GET/PATCH /agents; pause and guardrail toggles; approvals table and endpoints; Jira create on approve.
2d	Brief scheduler, Briefs page endpoints. Optional: server-side STT/TTS for the Voice call page.

CHECKPOINT · CP5 · Finance

- ✓ See 8.3.

CHECKPOINT · CP6 · HR

- ✓ See 8.4.

CHECKPOINT · CP7 · Executive and admin

- ✓ See 8.5, plus: pausing Tech on the Agents page sends Tech questions to Mentor; a Jira ticket is created only after approval, with an audit row.

CHECKPOINT · CP8 · Briefs and voice

- ✓ See Sections 9 and 11.

Phase 3 · Meetings · weeks 9–12

Astro in real calls, and one company using it every day.

Step	Build
3a	Calendar connector and sync; GET /meetings, invite switch, settings; bot provider adapter; join, introduce, leave.
3b	Streaming STT with speakers, speaker-to-user mapping, guest detection, wake word, spoken and chat answers, ask privately, live SSE stream.
3c	Notes step (decisions, actions), recap writer, recap endpoints, action approvals to Jira, Slack follow-up, retention job.
3d	Design-partner pilot: onboarding with the Setup flow, weekly review of feedback and failed questions.

CHECKPOINT · CP9 to CP11 · Meetings

- ✓ See Section 10.

CHECKPOINT · CP12 · Pilot

- ✓ One design-partner team has used Astro for at least two weeks on their own data.
- ✓ You have their accuracy numbers from feedback, their list of unanswered questions, and zero permission incidents.
- ✓ They agree to continue, and you know what they would pay (feeds the pitch deck).

Phase 4 · Harden and scale · after week 12

Things larger customers need.

Step	Build
4.1	SAML SSO and SCIM provisioning; group-based roles.
4.2	More connectors: Confluence, Slack search, Zendesk or Intercom, CRM, SharePoint; the Astro relay for private systems.
4.3	Zoom and Teams in meetings; your own bot or platform media APIs if costs demand it.
4.4	Dedicated-tenant and customer-cloud deployments with Terraform.
4.5	Security programme: penetration test, SOC 2 readiness, data-processing agreements, sub-processor list.
4.6	Presets and agents for finance, healthcare and retail companies (for example Risk and compliance, Operations, Sales).
4.7	Admin analytics: questions per team, unanswered topics, cost per company.

SECTION 16

Testing, evaluation and safety

Test	What it proves	When it runs
Unit tests	Tools, SQL gateway, chunking, parsers, approvals	Every pull request
Cross-tenant suite	No endpoint, tool or job returns another company's data	Every pull request
Permission suite	Role × domain and source ACLs hold for every agent, including Executive calling others	Every pull request
Prompt-injection suite	Documents with hidden instructions can't make an agent call tools, leak data or change its rules	Every pull request
Eval sets	Retrieval hit rate, answer correctness with a valid citation, routing accuracy	Nightly and before each release
Meeting replay	Recorded test meetings produce the same decisions and actions	Before each meetings release
Load test	Streaming stays smooth at your target concurrent users	Before each design partner joins

Quality signals to watch in production

- Thumbs-down rate per agent and per company.
- Questions answered “I couldn't find this”: usually missing or unconnected documents, which is a to-do list for the admin.
- Answers whose citations failed the verify step.
- Denied-access events: many of them can mean the matrix is too strict, or that someone is probing.

SECTION 17

Risks and how to handle them

Risk	Why it matters	What to do
Data leaks between users or companies	One incident ends trust with a customer	RLS, three filters on every read, permission suites in CI, audit log, no admin “see everything” mode
Wrong answers stated confidently	People act on them	Answer only from context, verify citations, show sources, easy feedback, eval sets
Prompt injection through documents	A doc could try to steer the agent	Context treated as data, tool allowlists in code, approvals for every write
Meeting bots break when platforms change	Meetings stop working overnight	Hosted provider first, one adapter interface, monitoring on join success
Recording consent	Legal exposure for the customer and you	Announce on join, guest rules, admin controls, retention, no audio stored by default
Model and speech costs	Margins disappear with heavy use	Small model for routing, caching, per-company caps, usage pricing for calls
Scope creep	Five agents and meetings is a lot for a small team	Hold to the checkpoints; don’t start Phase 3 until CP4 passes
Connector sprawl	Every customer asks for a different tool	Two connectors per preset first; a clean connector interface; the relay for the rest

APPENDIX A

API endpoints

Everything below lives under `/api` and is scoped to the signed-in user's company. Endpoints marked “exists in UI” are already called by the frontend.

Endpoint	Purpose	Status
GET <code>/me</code>	Current user, company, roles	To add
GET <code>/auth/{provider}/start</code> , <code>/callback</code> ; POST <code>/auth/logout</code>	Sign-in and out	To add
POST <code>/chat/stream</code>	Ask, SSE events route/agent/token/sources/done/error	Exists in UI
GET <code>/conversations</code> , <code>/conversations/{id}</code>	Chat history	To add
GET <code>/companies/{id}/sources</code>	Knowledge sources and status	Exists in UI
GET <code>/connectors/{kind}/start</code> , <code>/callback</code> ; DELETE <code>/sources/{id}</code>	Connect and disconnect a source	To add
PUT <code>/companies/{id}/access</code>	Role × domain matrix	Exists in UI
GET <code>/audit</code>	Access checks	To add
GET <code>/companies/{id}/agents</code> ; PATCH <code>.../agents/{agentId}</code>	Agent configs, pause, guardrails	Exists in UI
GET <code>/companies/{id}/meetings</code>	Calendar meetings	Exists in UI
POST <code>/companies/{id}/meetings/join</code>	Send Astro by link	Exists in UI
PATCH <code>/companies/{id}/meetings/{meetingId}</code>	Invite switch	Exists in UI
PUT <code>/companies/{id}/meeting-settings</code>	In-call behaviour	Exists in UI
GET <code>/meetings/{id}/live</code>	Live SSE stream	Exists in UI
PATCH <code>/meetings/{id}/live</code> ; POST <code>.../ask-private</code> ; POST <code>.../leave</code>	Live controls	Exists in UI
GET <code>/meetings/{id}/recap</code>	Recap	Exists in UI
POST <code>/meetings/{id}/actions/{actionId}/approve</code> ; POST <code>.../follow-up</code>	Approvals, Slack follow-up	Exists in UI
GET <code>/briefs</code> , <code>/briefs/{id}</code> ; PUT <code>/brief-schedules</code>	Briefs	To add
POST <code>/feedback</code>	Thumbs up or down on an answer	To add

APPENDIX B

Database schema

The core tables, trimmed to the columns that matter. Every table below except companies has `company_id` and an RLS policy.

```
companies      (id, name, domain_preset, email_domain, retention_days, created_at)
users          (id, company_id, email, name, status, created_at)
roles          (id, company_id, name)                                -- member, lead, hr, finance, leadership, admin
user_roles     (user_id, role_id)
groups         (id, company_id, external_id, name)                  -- imported from Google / Entra
user_groups    (user_id, group_id)
role_access    (company_id, role_id, domain, allowed)              -- the Knowledge page matrix
linked_accounts (id, company_id, user_id, provider, external_user_id, token_ref)

sources        (id, company_id, kind, name, status, last_synced_at, token_ref, settings_json)
documents      (id, company_id, source_id, external_id, title, url, domain, author,
               updated_at, acl_users uuid[], acl_groups uuid[], acl_public_in_company bool)
chunks         (id, company_id, document_id, domain, heading_path, text, tsv, embedding, content_hash)

agent_configs  (company_id, agent_id, paused, guardrails_json, tools_json, roles_json)
conversations  (id, company_id, user_id, title, created_at)
messages       (id, company_id, conversation_id, role, agent_id, text, sources_json, created_at)
pending_actions (id, company_id, requested_by, kind, payload_json, status, approved_by, result_json)
audit_log      (id, company_id, user_id, event, agent_id, detail_json, created_at)

meetings       (id, company_id, platform, external_id, title, starts_at, ends_at,
               astro_invited, has_guests, bot_session_id, status)
meeting_events (id, company_id, meeting_id, type, payload_json, at)  -- transcript, answer, decision, action
recaps         (meeting_id, company_id, summary, decisions_json, follow_up_json, created_at)
action_items   (id, company_id, meeting_id, task, owner_user_id, due, destination, status, external_ref)

briefs         (id, company_id, type, owner_user_id, content_json, sources_json, created_at)
brief_schedules (id, company_id, type, cron, owner_user_id, audience_json)
usage          (company_id, day, llm_tokens, embed_tokens, stt_minutes, cost_estimate)
```

NOTE · Last word

- Build in the order of the checkpoints, keep the permission tests green, and show every step to a real company as early as possible. The UI is already ahead of the backend, which means every checkpoint you pass shows up on screen straight away.